

PART XIX

ADVANCED ALGORITHMS

Divide and conquer, meet in the middle, sweep line, small-to-large, and rerooting

This Part collects a set of general-purpose techniques that don't belong to any single data structure — they're ways of restructuring a problem so that an existing tool (sorting, a Fenwick tree, DSU, brute force) becomes fast enough. Binary lifting and LCA were covered alongside the other advanced data structures; bitmask DP was covered in the Dynamic Programming Part. What's left here is divide and conquer, meet in the middle, coordinate compression, sweep line and offline queries, the small-to-large technique, rerooting (promised back in the Dynamic Programming Part), monotonic stacks/queues, and a short introduction to randomization.

Contents of Part XIX

- 1. Divide and Conquer
- 2. Meet in the Middle
- 3. Coordinate Compression
- 4. Sweep Line
- 5. Offline Queries
- 6. Small-to-Large (DSU on Tree)
- 7. Rerooting Technique
- 8. Monotonic Stack and Monotonic Queue
- 9. Randomized Algorithms — An Introduction
- Part XIX Revision, Cheat Sheet, and Interview Questions

Chapter 1 — Divide and Conquer

In one sentence: divide and conquer solves a problem by splitting it into smaller independent subproblems, solving each recursively, and combining their answers — the difference from dynamic programming is that the subproblems here don't overlap, so nothing needs to be cached.

You've already used this paradigm: merge sort splits an array in half, sorts each half recursively, and merges the results; quicksort partitions around a pivot and recurses on each side. This chapter treats the paradigm explicitly and works through a less obvious application.

1.1 The three-step recipe

- 1 Divide — split the problem into smaller subproblems of the same type.
- 2 Conquer — solve each subproblem recursively (base case: solve directly when small enough).
- 3 Combine — merge the subproblem answers into the answer for the original problem.

1.2 Example: maximum subarray sum via divide and conquer

This problem has an $O(n)$ single-pass solution (Kadane's algorithm), but the divide-and-conquer version is a standard exercise for the paradigm itself, and the “combine” step it needs — handling a subarray that crosses the midpoint — is a recurring idea in this family of algorithms.

```

int crossSum(vector<int> &a, int lo, int mid, int hi) {
    int leftSum = INT_MIN, sum = 0;
    for (int i = mid; i >= lo; i--) { sum += a[i]; leftSum =
max(leftSum, sum); }
    int rightSum = INT_MIN; sum = 0;
    for (int i = mid + 1; i <= hi; i++) { sum += a[i]; rightSum =
max(rightSum, sum); }
    return leftSum + rightSum; // best subarray that crosses the
midpoint
}

int maxSubarrayDC(vector<int> &a, int lo, int hi) {
    if (lo == hi) return a[lo];
    int mid = lo + (hi - lo) / 2;
    int leftBest = maxSubarrayDC(a, lo, mid);
    int rightBest = maxSubarrayDC(a, mid + 1, hi);
    int crossBest = crossSum(a, lo, mid, hi);
    return max({leftBest, rightBest, crossBest});
}

```

The best subarray either lies entirely in the left half, entirely in the right half, or straddles the midpoint — those three cases are exhaustive, which is exactly what makes the combine step correct. Complexity: $T(n) = 2T(n/2) + O(n)$, which solves to $O(n \log n)$ — worse than Kadane's $O(n)$, but the technique generalizes to problems (like counting inversions, or the closest-pair-of-points problem) that don't have a simpler linear solution.

1.3 The Master Theorem, informally

For a recurrence of the shape $T(n) = a \cdot T(n/b) + O(n^d)$, the Master Theorem gives the closed-form complexity directly from a , b , and d , without solving the recurrence by hand each time.

Compare d to $\log^b(a)$	Result
$d > \log^b(a)$	$O(n^d)$ — the combine step dominates
$d = \log^b(a)$	$O(n^d \log n)$ — balanced (this is merge sort's case: $a=2$, $b=2$, $d=1$)
$d < \log^b(a)$	$O(n^{\log^b(a)})$ — the recursive branching dominates

♦ Practical rule

You rarely need the Master Theorem's full derivation in a contest — what matters is recognizing the shape $T(n) = a \cdot T(n/b) + O(n^d)$ in your own recursive solution and knowing which regime it falls into, so you can sanity-check your complexity against the given constraints before submitting.

Practice — Chapter 1**Problem 19.1 — Count Inversions [Level 4]**

Hint 1: An inversion is a pair (i, j) with $i < j$ and $a[i] > a[j]$.

Key observation: Merge sort's merge step already compares elements from the left and right halves in order — count cross-inversions during that merge instead of after.

Approach: Modify merge sort: whenever an element from the right half is placed before remaining elements of the left half, add the count of those remaining left elements to the inversion total.

Complexity: $O(n \log n)$.

Chapter 2 — Meet in the Middle

In one sentence: when brute force over all 2^n subsets is too slow but n is still moderate (roughly $n \leq 40$), split the set in half, brute-force each half separately ($2^{(n/2)}$ each), and combine the two halves' results efficiently.

This is the exact “ $n \leq 40 \rightarrow$ meet in the middle” row from the constraints-to-complexity table introduced back in the Master Roadmap: 2^{40} is far too slow, but 2×2^{20} (roughly 2 million) is fast.

2.1 Example: subset sum with n up to 40

Given up to 40 numbers, determine if any subset sums to exactly a target T .

```

bool subsetSumMITM(vector<long long> &a, long long target) {
    int n = a.size();
    int half = n / 2;
    vector<long long> left(a.begin(), a.begin() + half);
    vector<long long> right(a.begin() + half, a.end());

    auto allSums = [](vector<long long> &v) {
        int m = v.size();
        vector<long long> sums;
        for (int mask = 0; mask < (1 << m); mask++) {
            long long s = 0;
            for (int i = 0; i < m; i++) if (mask & (1 << i)) s += v[i];
            sums.push_back(s);
        }
        return sums;
    };

    vector<long long> leftSums = allSums(left);
    vector<long long> rightSums = allSums(right);
    sort(rightSums.begin(), rightSums.end());

    for (long long ls : leftSums) {
        if (binary_search(rightSums.begin(), rightSums.end(), target - ls))
            return true;
    }
    return false;
}

```

Complexity: $O(2^{(n/2)} \times (n/2))$ to generate each half's sums, plus $O(2^{(n/2)} \log(2^{(n/2)}))$ to sort and $O(2^{(n/2)} \log(2^{(n/2)}))$ to binary search — overall roughly $O(2^{(n/2)} \times n)$, which is enormously faster than the $O(2^n)$ full brute force for n around 40.

✦ How to recognize it

Clue combination: the constraint is roughly $20 < n \leq 40$ (too big for direct 2^n enumeration, too small to obviously need a polynomial algorithm), and the problem is naturally phrased over subsets. This narrow constraint band is close to a direct signal for meet in the middle.

Practice — Chapter 2

Problem 19.2 — Closest Subset Sum to Target [Level 5]

Hint 1: n is up to about 40, ruling out direct subset enumeration.

Approach: Generate and sort both halves' subset sums; for each left-half sum, binary search the right half for the closest complement to the target.

Complexity: $O(2^{(n/2)} \times n)$.

Chapter 3 — Coordinate Compression

In one sentence: when values are large or sparse but only their relative order matters, replace each value with its rank among all distinct values that appear — shrinking the value range to at most n , so array-indexed structures (Fenwick tree, difference arrays) become usable.

```
vector<int> compress(vector<int> &a) {
    vector<int> sorted_vals = a;
    sort(sorted_vals.begin(), sorted_vals.end());
    sorted_vals.erase(unique(sorted_vals.begin(),
                             sorted_vals.end()), sorted_vals.end());

    vector<int> compressed(a.size());
    for (int i = 0; i < (int)a.size(); i++) {
        // rank of a[i] among distinct sorted values, 0-indexed
        compressed[i] = lower_bound(sorted_vals.begin(),
                                     sorted_vals.end(), a[i])
                        - sorted_vals.begin();
    }
    return compressed;
}
```

Complexity: $O(n \log n)$ for the sort and the lookups. This is almost never the final answer to a problem — it's a preprocessing step that makes a Fenwick tree, segment tree, or difference array applicable to values up to 10^9 by remapping them into the range $[0, n)$.

♦ How to recognize it

Clue combination: the problem needs an array-indexed structure (BIT, segment tree, difference array) but the actual values are far too large or sparse to index directly, and the problem only ever compares values or counts things “less than” or “greater than” a given value — never needs the value's exact magnitude for arithmetic.

Practice — Chapter 3**Problem 19.3 — Count Smaller Elements to the Right [Level 4]**

Hint 1: Values can be as large as 10^9 , but there are at most n of them.

Approach: Coordinate-compress the values, then process the array right to left, querying and updating a Fenwick tree indexed by compressed rank.

Complexity: $O(n \log n)$.

Chapter 4 — Sweep Line

In one sentence: convert a geometric or interval problem into a sequence of “events” sorted by position, then process events left to right, maintaining a running data structure that reflects only what's currently “active.”

Part XIV's minimum-meeting-rooms solution was already a sweep line in disguise: “meeting starts” and “meeting ends” are events, sorted and processed in order, with a running room counter as the active state. This chapter names the pattern and extends it to rectangles.

4.1 Example: total area covered by overlapping rectangles

Given axis-aligned rectangles, find the total area covered by their union (overlaps not double-counted). Sweep a vertical line left to right over the x -coordinates where any rectangle starts or ends; between

consecutive x-events, the set of “active” y-intervals is fixed, so the covered height in that x-slice can be computed by merging active y-intervals.

```
// events: {x, y1, y2, +1 for rectangle start, -1 for rectangle
end}
struct Event { long long x; int y1, y2, type; };

long long unionArea(vector<array<long long,4>> &rects) { // {x1,
y1, x2, y2}
    vector<Event> events;
    for (auto &[x1, y1, x2, y2] : rects) {
        events.push_back({x1, (int)y1, (int)y2, +1});
        events.push_back({x2, (int)y1, (int)y2, -1});
    }
    sort(events.begin(), events.end(), [](auto &a, auto &b) { return
a.x < b.x; });

    long long totalArea = 0;
    long long prevX = events.empty() ? 0 : events[0].x;
    multiset<pair<int,int>> activeIntervals; // simplification: see
note below
    for (auto &e : events) {
        // covered height = union length of all currently active
y-intervals
        long long width = e.x - prevX;
        long long coveredHeight = 0; // computed by merging
activeIntervals (see note)
        totalArea += width * coveredHeight;
        if (e.type == 1) activeIntervals.insert({e.y1, e.y2});
        else activeIntervals.erase(activeIntervals.find({e.y1, e.y2}));
        prevX = e.x;
    }
    return totalArea;
}
```

Implementation note

Computing 'covered height' efficiently on every event usually needs a segment tree over compressed y-coordinates that supports range add and 'total covered length' queries — this combines three techniques from this book at once: coordinate compression (Chapter 3), sweep line (this chapter), and a segment tree (Advanced Data Structures). The simplified multiset sketch above shows the event structure; a full solution replaces the 'computed by merging' line with such a segment tree.

Complexity: $O(n \log n)$ for sorting events, plus $O(\log n)$ per segment-tree update per event — $O(n \log n)$ overall in the full implementation.

Practice — Chapter 4**Problem 19.4 — The Skyline Problem [Level 5]**

Hint 1: Each building contributes a 'start' event (height increases) and an 'end' event (height may decrease).

Approach: Sweep over x-coordinates; maintain a multiset of active building heights, and emit a skyline point whenever the maximum active height changes.

Complexity: $O(n \log n)$.

Chapter 5 — Offline Queries

In one sentence: when queries don't need to be answered in the order they're given, read all queries first, reorder them to make processing easier (often by sorting), and answer them in that more convenient order instead.

This is only valid when a query's answer doesn't depend on real-time interaction — i.e. no query result determines what a later query even asks. Whenever that condition holds, offline processing frequently turns an otherwise-hard problem into a straightforward sweep with a Fenwick tree or DSU.

5.1 Example: range sum queries with point updates, given in advance

If you already know every query (both updates and range-sum requests) before answering any of them, sort queries by their right endpoint and sweep a Fenwick tree left to right, answering each range query the moment the sweep passes its right endpoint — this is the same idea used for “count smaller elements to the right” in Chapter 3, generalized to arbitrary offline range questions.

5.2 Example: offline DSU for connectivity-over-time queries

Given a sequence of edge additions and “are u and v connected?” queries, if all queries are known upfront, DSU (Part XIII) directly answers each connectivity query in $O(\alpha(n))$ by processing edges and queries together in their given order — no special reordering needed here, but the same “know everything in advance” requirement applies. A more advanced variant — queries that ask about connectivity at some point in the past, after later edges have already been added — needs an offline approach that processes queries in a different order than they were given (often reverse chronological order with a “DSU with rollback”), which is why this pattern is grouped under offline queries rather than plain DSU usage.

♦ How to recognize it

Clue phrases: “you are given q queries in advance” combined with any structure that's easy to build incrementally but hard to answer in arbitrary jump-around order (Fenwick tree, DSU, sorted sweep). If reordering the queries would make the problem easier and the problem doesn't require live/interactive answers, it's a signal to go offline.

Practice — Chapter 5

Problem 19.5 — Range Sum Queries with Point Updates (offline) [Level 4]

Hint 1: All updates and queries are given in advance, not interactively.

Approach: Sort queries by right endpoint; sweep a Fenwick tree left to right, applying updates as you pass them and answering each query when the sweep reaches its right endpoint.

Complexity: $O((n + q) \log n)$.

Chapter 6 — Small-to-Large (DSU on Tree)

In one sentence: when merging two data structures (like sets of colors in subtrees), always merge the smaller one into the larger one — this simple rule guarantees that any single element is moved at most $O(\log n)$ times across the whole algorithm, even though any individual merge can look expensive.

This is the same insight behind DSU's union-by-size (Part XIII), applied directly as a technique rather than hidden inside a structure.

6.1 Why $O(\log n)$ amortized merges per element

Every time an element is moved during a merge, it's moved into a structure at least twice the size of the one it came from (small-to-large guarantees the destination is at least as large as the source, and typically far larger). An element can only double in “container size” at most $\log_2(n)$ times before it belongs to the entire structure — so no element is ever moved more than $O(\log n)$ times in total, giving $O(n \log n)$ total work across all merges combined, not $O(n^2)$.

6.2 Example: count distinct colors in every subtree

Given a tree where every node has a color, compute for every node the number of distinct colors in its subtree. Small-to-large: process children first (post-order), keep each child's color set as a hash set,

and merge all children's sets into the largest child's set (rather than always creating a fresh set).

```
vector<vector<int>> adj;
vector<int> color;
vector<unordered_set<int>*> nodeSet;
vector<int> answer;

void dfs(int u, int parent) {
    nodeSet[u] = new unordered_set<int>();
    nodeSet[u]->insert(color[u]);
    for (int v : adj[u]) {
        if (v == parent) continue;
        dfs(v, u);
        if (nodeSet[v]->size() > nodeSet[u]->size())
            swap(nodeSet[u], nodeSet[v]); // keep the LARGER set as u's set
        for (int c : *nodeSet[v]) nodeSet[u]->insert(c); // merge the
        smaller in
        delete nodeSet[v];
    }
    answer[u] = nodeSet[u]->size();
}
```

Complexity: $O(n \log n)$ total across the whole tree, even though a naive-looking reading of the code (“merge sets at every node”) might suggest $O(n^2)$ — the small-to-large guarantee is exactly what brings it down.

♦ How to recognize it

Clue combination: a tree (or a recursive partition) where you need to merge child results (sets, counts, frequency maps) at every node, and a naive merge-everything-every-time approach looks like it should be too slow. Small-to-large is the fix whenever “merge the smaller into the larger” is a valid operation for the data structure involved.

Practice — Chapter 6

Problem 19.6 — Most Frequent Color in Every Subtree [Level 5]

Hint 1: You need, per node, the color (or colors) with the maximum count within that subtree.

Approach: Small-to-large merging of per-node frequency maps, tracking the running maximum frequency and which colors achieve it as sets are merged.

Complexity: $O(n \log n)$.

Chapter 7 — Rerooting Technique

Part XV (Dynamic Programming) previewed this: some tree problems need the answer computed for every possible root, not just one fixed root. Recomputing a full $O(n)$ tree DP separately for each of the n possible roots costs $O(n^2)$. Rerooting brings this down to $O(n)$ total.

7.1 The two-pass idea

- 1 First DFS (down) — root the tree arbitrarily (say, at node 0) and compute each node's answer considering only its own subtree, exactly like ordinary tree DP.
- 2 Second DFS (down again, reusing the parent's already-rerooted answer) — for each child, derive what the child's answer WOULD be if the tree were rooted there, using only the parent's answer and the child's own subtree answer — no other recomputation.

The key requirement: step 2 must be able to “undo” the child's contribution from the parent's answer and then “redo” the parent's contribution onto the child, using $O(1)$ or $O(\log n)$ work — not by recomputing from scratch.

7.2 Example: sum of distances to all other nodes, for every node

For every node, compute the sum of distances from that node to every other node in the tree.

```

vector<vector<int>> adj;
vector<long long> subtreeSize, downSum, answer;

void dfsDown(int u, int parent) {
    subtreeSize[u] = 1;
    downSum[u] = 0;
    for (int v : adj[u]) {
        if (v == parent) continue;
        dfsDown(v, u);
        subtreeSize[u] += subtreeSize[v];
        downSum[u] += downSum[v] + subtreeSize[v]; // each node in v's
        subtree is 1 edge farther
    }
}

void reroot(int u, int parent, int n) {
    for (int v : adj[u]) {
        if (v == parent) continue;
        // moving the root from u to v:
        // - every node in v's subtree gets 1 CLOSER
        // - every other node (n - subtreeSize[v] of them) gets 1 FARTHER
        answer[v] = answer[u] - subtreeSize[v] + (n - subtreeSize[v]);
        reroot(v, u, n);
    }
}

// main: dfsDown(0, -1); answer[0] = downSum[0]; reroot(0, -1, n);

```

The rerooting step is exactly the “undo/redo” idea: `subtreeSize[v]` nodes move one step closer, and all `n - subtreeSize[v]` other nodes move one step farther — both quantities are already known from the first pass, so each reroot step is $O(1)$. Complexity: $O(n)$ total for both passes, instead of $O(n^2)$ for recomputing every root from scratch.

✦ How to recognize it

Clue phrase: “for every node, compute [some tree quantity] as if that node were the root.” If a single-root tree DP would naturally solve the problem for one fixed root, rerooting almost always extends it to all roots in the same total time.

Practice — Chapter 7

Problem 19.7 — Sum of Distances in Tree [Level 5]

Hint 1: Solve for one root first with a normal subtree-size DFS.

Approach: Use the two-pass rerooting shown above: down-pass computes subtree sizes and one root's answer; the reroot pass derives every other node's answer in $O(1)$ each.

Complexity: $O(n)$.

Chapter 8 — Monotonic Stack and Monotonic Queue

A quick, practical treatment of a pattern referenced earlier (Part IX introduced monotonic stacks alongside plain stacks) — collected here as a technique in its own right, since it recurs constantly across very different-looking problems.

8.1 Monotonic stack: next greater element

In one sentence: maintain a stack of indices whose values are in decreasing order; whenever a new value is bigger than the stack's top, that top element's “next greater element” has just been found, so pop it and repeat.

```
vector<int> nextGreaterElement(vector<int> &a) {
    int n = a.size();
    vector<int> result(n, -1);
    stack<int> st; // stores indices, values at those indices are
    decreasing
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            result[st.top()] = a[i];
            st.pop();
        }
        st.push(i);
    }
    return result; // remaining stack entries have no next greater
    element (-1)
}
```

Why is this $O(n)$ despite the nested-looking while loop? Every index is pushed exactly once and popped at most once across the ENTIRE run — the total work done by all the while-loop pops, summed over every iteration of the outer loop, is bounded by n , not n^2 . This is the same amortized-cost reasoning used throughout this book (DSU, small-to-large) applied to a stack instead.

8.2 Monotonic queue: sliding window maximum

Find the maximum of every contiguous window of size k in an array, in $O(n)$ total instead of $O(n \times k)$. Maintain a deque of indices whose values are decreasing; the front of the deque is always the maximum of the current window.

```
vector<int> slidingWindowMax(vector<int> &a, int k) {
    deque<int> dq; // stores indices, values at those indices are
    decreasing
    vector<int> result;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front(); //
        out of window
        while (!dq.empty() && a[dq.back()] < a[i]) dq.pop_back(); //
        maintain decreasing
        dq.push_back(i);
        if (i >= k - 1) result.push_back(a[dq.front()]);
    }
    return result;
}
```

Complexity: $O(n)$ — the same amortized argument as the monotonic stack applies to both ends of the deque. This exact structure is also called “deque optimization” when it appears inside a DP transition to keep a sliding-window maximum/minimum available in $O(1)$ per state instead of $O(k)$.

♦ How to recognize it

Clue phrases: “next greater/smaller element,” “maximum/minimum in every sliding window,” or a DP transition that needs “the best value among the last k states.” All three point to a monotonic stack or monotonic queue instead of a naive $O(n^2)$ or $O(n \cdot k)$ scan.

Practice — Chapter 8

Problem 19.8 — Largest Rectangle in a Histogram [Level 4]

Hint 1: For each bar, the largest rectangle using that bar's height extends until a shorter bar appears on either side.

Approach: Monotonic increasing stack of indices; when a shorter bar is found, pop and compute the area using the popped bar's height and the current span.

Complexity: $O(n)$.

Chapter 9 — Randomized Algorithms — An Introduction

In one sentence: a randomized algorithm makes some of its decisions using random choices, trading a guaranteed worst case for an expected (average-case) guarantee that holds regardless of the specific input — including inputs an adversary designed specifically to break a deterministic version.

9.1 Randomized quickselect / quicksort pivot choice

Quicksort's worst case ($O(n^2)$) happens when the pivot is always the smallest or largest remaining element — which is easy for an adversary to construct if the pivot choice is deterministic (e.g. always the first element) and they know your code. Choosing the pivot uniformly at random makes the worst case depend on an extremely unlikely sequence of random draws rather than on the input itself, giving expected $O(n \log n)$ regardless of input order.

```
int randomizedPartition(vector<int> &a, int lo, int hi) {
    int randIdx = lo + rand() % (hi - lo + 1);
    swap(a[randIdx], a[hi]); // move random pivot to the end, then
    partition normally
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) swap(a[i++], a[j]);
    }
    swap(a[i], a[hi]);
    return i;
}
```

9.2 Why competitive programmers randomize against hacks

Codeforces problems are sometimes “hackable” — other contestants can submit a specific input designed to break a known weakness in a common approach (e.g. using an `unordered_map` with a naive hash on adversarially-chosen keys, which can be forced into $O(n)$ per operation instead of $O(1)$). Two common defenses: shuffle the input randomly before processing (removes any adversarial ordering), and add a random additive constant to hash keys before hashing them.

```
// defend against anti-hash-table hacks on unordered_map<int,int>
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count())
;
int salted(int x) { return x ^ rng(); } // scramble keys before
using them as hash input
```

Scope note

This chapter is an introduction, not a full treatment — randomized data structures like treaps, and randomized algorithms with formally proven expected bounds (e.g. randomized min-cut), are specialized topics beyond this book's scope. The two takeaways worth internalizing here are: randomization can neutralize adversarial worst cases, and it costs almost nothing to add as defensive practice (random pivot, shuffled input, salted hashes).

Part XIX — Master Revision

Must Remember

- Divide and conquer splits into independent (non-overlapping) subproblems; the Master Theorem classifies $T(n) = a \cdot T(n/b) + O(n^d)$ into one of three complexity regimes.
- Meet in the middle applies when n is too large for full 2^n brute force but small enough to brute-force each half (roughly $n \leq 40$).
- Coordinate compression remaps large/sparse values to ranks in $[0, n)$ so array-indexed structures become usable — valid whenever only relative order matters.
- Sweep line converts a geometric/interval problem into sorted events processed left to right with a running active-state structure.
- Offline processing (reordering queries, often by sorting) is valid exactly when no query's answer affects what a later query asks.
- Small-to-large guarantees $O(n \log n)$ total merge work by always merging the smaller structure into the larger one — no element moves more than $O(\log n)$ times.
- Rerooting extends a single-root tree DP to all roots in $O(n)$ total, by deriving each child's rerooted answer from its parent's in $O(1)$.
- Monotonic stacks/queues answer 'next greater/smaller' and 'sliding window max/min' in $O(n)$ total via amortized analysis — each element is pushed and popped at most once.
- Randomization (random pivots, shuffled input, salted hashes) defends against adversarially-constructed worst-case inputs at almost no extra cost.

Common Mistakes Recap

Mistake	Fix
Using meet in the middle when n is small enough for direct DP/brute force, or too large even for $2^{(n/2)}$	Reserve it for the roughly $20 < n \leq 40$ band where neither extreme applies

Mistake	Fix
Coordinate-compressing when the problem needs actual value magnitudes (e.g. summing the values themselves)	Only compress when the algorithm only needs relative order, not the values themselves
Processing sweep-line events without a stable tie-break rule (e.g. processing an 'end' event after a 'start' event at the same coordinate when it should be the reverse)	Explicitly decide and code the tie-break order for simultaneous events
Assuming offline reordering is always valid	Check that no query's answer influences a later query before reordering
Always merging into a fixed structure instead of the larger one in small-to-large	Compare sizes explicitly and merge the smaller into the larger every time
Recomputing a full tree DP separately for every root	Use rerooting for $O(n)$ total instead of $O(n^2)$

Complexity Table — Part XIX

Technique	Typical Complexity
Divide and conquer (general)	$T(n) = a \cdot T(n/b) + O(n^d)$ — see Master Theorem
Meet in the middle	$O(2^{n/2} \times n)$
Coordinate compression	$O(n \log n)$
Sweep line (with a segment tree)	$O(n \log n)$
Offline queries (sort + Fenwick sweep)	$O((n + q) \log n)$
Small-to-large merging	$O(n \log n)$ total
Rerooting	$O(n)$ total
Monotonic stack / queue	$O(n)$ amortized

Interview-Style Questions

- 1 What distinguishes divide and conquer from dynamic programming, given that both use recursion?

- 2 Why does meet in the middle only make sense in a specific constraint band, rather than for all n ?
- 3 Why is coordinate compression only valid when a problem doesn't need the values' actual magnitudes?
- 4 Why is small-to-large merging $O(n \log n)$ total, even though a single merge step can look expensive?
- 5 Why does rerooting need only $O(1)$ work per node to convert one root's answer into a neighbor's, in the sum-of-distances example?
- 6 Why is a monotonic stack's next-greater-element algorithm $O(n)$ despite its nested-looking while loop?
- 7 Why does randomizing quicksort's pivot choice defend against adversarially chosen inputs?

Cheat Sheet — Part XIX

```
// Divide and conquer:  $T(n) = a \cdot T(n/b) + O(n^d)$  -- classify via Master Theorem
// Meet in the middle:  $n \leq \sim 40$ , split in half, brute-force + sort + binary search
// Coordinate compression: sort + unique + lower_bound  $\Rightarrow$  ranks in  $[0, n)$ 
// Sweep line: sort events by position, maintain running active-state structure
// Offline queries: sort/reorder queries when future queries don't depend on them
// Small-to-large: always merge the SMALLER structure into the LARGER one
// Rerooting: dfsDown() for one root's answer, then reroot() derives every other
// node's answer in  $O(1)$  using subtree size / count bookkeeping
// Monotonic stack: push/pop while maintaining increasing or decreasing order
// Monotonic queue (deque): pop_front stale indices, pop_back weaker values
// Randomization: random pivot, mt19937 for shuffling, salted keys vs hash-hacks
```

 Coming up in Part XX

Part XX — C++ OOP / University Module steps away from competitive algorithms to cover the university-syllabus side of this book properly: classes and objects, constructors and destructors, operator overloading, inheritance, polymorphism, and templates — taught thoroughly, but kept as a clearly separate module rather than the book's main focus, exactly as the original course spec asked for.